MPAS-URBAN QUICKSTART | GUIDE 1 OF 3

Install HKUST-MPAS

Start from the official HKUST-MPAS source, compile both atmospheric executables, and add the MPAS-Urban geographical data.

OUTCOME You will have a standard HKUST-MPAS installation for learning and new experiments, plus a clearly separated optional source tree for the HRAIN2025 NTDK cutoff experiment.

AUTHOR LIU Zhuo
AFFILIATION The Hong Kong University of Science and Technology (HKUST)

Before you begin

This guide supplements rather than replaces the official MPAS-Atmosphere tutorial. New users should first complete the official tutorial or Quick Start Guide and confirm that a standard MPAS case can be compiled and run.

- Official tutorial: https://mpas-dev.github.io/atmosphere/tutorial.html
- Quick Start Guide: https://www2.mmm.ucar.edu/projects/mpas/site/documentation/users_guide/quick_start.html

Use a Unix/Linux server. On an HPC system, prefer the compiler, MPI, NetCDF, and PnetCDF modules supported by the site. Keep one compiler/MPI family for MPAS and every MPI-linked library.

If the server does not yet provide a working MPAS build environment, first read the dependency sections of the official tutorial and then use Appendix A in this guide. Appendix A gives one complete user-space GNU/Open MPI example for a machine without an existing MPAS environment. On a managed HPC cluster, site-provided modules remain the preferred route.

Step 1 | Check the existing MPAS build environment

COMMAND

```
command -v gcc
command -v gfortran
command -v mpicc
command -v mpif90
command -v nc-config
command -v nf-config
command -v pnetcdf-config

mpicc --showme:command 2>/dev/null || mpicc -show
mpif90 --showme:command 2>/dev/null || mpif90 -show
```

CHECK For the gfortran build target, the MPI wrappers should resolve to a compatible GNU compiler family. Stop and correct the module environment if the wrappers point to a different compiler family.

ONE ROOT FOR THE COMPLETE TUTORIAL

All local source trees, dependencies, datasets, case files, and model output are kept under one root. The three guides use MPAS_ROOT so that new users do not have to manage unrelated directories across $HOME.

```
$HOME/MPAS/
|-- HKUST-MPAS/
|-- HKUST-MPAS-NTDK/            # optional HRAIN2025 cutoff build
|-- DATA/mpas_static/
|   |-- mpas_cglc_lcz/
|   `-- bnu_soiltype_top/       # HRAIN2025 cutoff build only
|-- MPAS-Urban-HRAIN2025/
|-- HRAIN2025_INPUT/
`-- HRAIN2025_30km_500m/
```

Step 2 | Download the standard HKUST-MPAS source

Use the HKUST-MPAS team repository and its hkust-dev branch as the default source for MPAS-Urban learning and new independent experiments.

STANDARD SOURCE

```
export MPAS_ROOT="$HOME/MPAS"
mkdir -p "$MPAS_ROOT"
cd "$MPAS_ROOT"

git clone --depth 1 --branch hkust-dev --single-branch   https://github.com/HKUST-MPAS/HKUST-
MPAS.git

cd HKUST-MPAS
git branch --show-current
test -f Makefile
test -d src/core_atmosphere
```

EXPECTED git branch --show-current prints hkust-dev.

Step 3 | Optional HRAIN2025 NTDK cutoff source

Skip this step for a standard MPAS-Urban installation. The following repository is a case-author derivative used for the HRAIN2025 cutoff experiment; it is not the official HKUST-MPAS main branch and does not represent the default HKUST-MPAS physics configuration.

CASE-AUTHOR DERIVATIVE

```
export MPAS_ROOT="$HOME/MPAS"
cd "$MPAS_ROOT"

git clone --depth 1 \
  --branch ntdk-grid-cutoff \
  --single-branch \
  https://github.com/Liuzh223/HKUST-MPAS-LIU.git \
  HKUST-MPAS-NTDK

cd HKUST-MPAS-NTDK
git branch --show-current

# Continue only if this prints: Pinned commit verified.
test "$(git rev-parse HEAD)" = \
  "7c1610b8f41781c2433f780bd7496da63b25a920" && echo "Pinned commit verified."
```

REPOSITORY NOTE HKUST-MPAS-LIU is a user-maintained derivative for this case, not the official HKUST-MPAS main repository. The ntdk-grid-cutoff branch is maintained for HRAIN2025. There is no separate terminal command named cutoff.

SOURCE BASE The pinned HRAIN2025 derivative remains based on MPAS-Atmosphere v8.2.2. Its selectable BNU soil-category input is a targeted backport of official MPAS development for v8.3, not a whole-tree upgrade to v8.3.

OPTION NOTE This derivative adds config_cu_disable_dx. The value is selected later in Guide 3; it is not a compile command.

Step 4 | Compile both MPAS executables

The build block reads the active NetCDF-C, NetCDF-Fortran, and PnetCDF prefixes from their configuration tools. This works with compatible site modules or the single-prefix stack in Appendix A and avoids copying a machine-specific path. This tutorial uses the SMIOL library bundled with the source, so unset PIO prevents an old shell setting from selecting an unintended external ParallelIO installation; it does not uninstall or delete PIO.

BUILD ENVIRONMENT

```
export MPAS_ROOT="$HOME/MPAS"

# Use the dependency stack currently active in this shell.
export NETCDF="$(nc-config --prefix)"
export NETCDFF="$(nf-config --prefix)"
export PNETCDF="$(pnetcdf-config --prefix)"

# Use the bundled SMIOL backend. This only clears a shell variable.
unset PIO

test -s "$NETCDF/include/netcdf.h"
test -s "$NETCDFF/include/netcdf.mod"
test -s "$PNETCDF/include/pnetcdf.h"

nc-config --version
nf-config --version
pnetcdf-config --version
```

The three guides define MPAS_BUILD as a reusable shell variable; it is not an official MPAS environment variable or namelist option. Point it to exactly one source directory here. After compilation, the same path supplies the executables and runtime files used by Guides 2 and 3.

```
# Standard HKUST-MPAS (default)
export MPAS_BUILD="$MPAS_ROOT/HKUST-MPAS"

# HRAIN2025 cutoff derivative (optional; use instead)
# export MPAS_BUILD="$MPAS_ROOT/HKUST-MPAS-NTDK"

cd "$MPAS_BUILD"
```

BUILD

```
make -j 4 gfortran CORE=init_atmosphere PRECISION=single
make clean CORE=atmosphere
make -j 1 gfortran CORE=atmosphere PRECISION=single
```

VERIFY

```
test -x init_atmosphere_model
test -x atmosphere_model
ls -lh init_atmosphere_model atmosphere_model
```

BUILD NOTE The serial atmosphere build avoids Fortran module races observed on some systems. Increase the parallel build count only after the toolchain has been verified.

Step 5 | Download the standard MPAS static data

DOWNLOAD

```
export MPAS_ROOT="$HOME/MPAS"
mkdir -p "$MPAS_ROOT/DATA"
cd "$MPAS_ROOT/DATA"

curl -fL -O   https://www2.mmm.ucar.edu/projects/mpas/mpas_static.tar.bz2

tar -xjf mpas_static.tar.bz2
test -d "$MPAS_ROOT/DATA/mpas_static"
```

Use the absolute path to $HOME/MPAS/DATA/mpas_static as config_geog_data_path during initialization.

Step 6 | Add the MPAS-Urban CGLC-LCZ data

DOWNLOAD AND EXTRACT

```
export MPAS_ROOT="$HOME/MPAS"
cd "$MPAS_ROOT/DATA"

curl -fL -O   https://github.com/HKUST-MPAS/HKUST-
MPAS/releases/download/LCZ_dataset_for_MPAS_Urban/mpas_cglc_lcz.tar.gz

mkdir -p "$MPAS_ROOT/DATA/mpas_static/mpas_cglc_lcz"
tar -xzf mpas_cglc_lcz.tar.gz   -C "$MPAS_ROOT/DATA/mpas_static/mpas_cglc_lcz"

test -s "$MPAS_ROOT/DATA/mpas_static/mpas_cglc_lcz/index"
```

The HKUST-MPAS source already supplies the MPAS-Urban parameter tables. Do not download replacement urban tables unless a specific code version instructs you to do so.

Step 7 | Add the BNU soil categories for HRAIN2025

Skip this step for the standard hkust-dev build. The pinned HRAIN2025 cutoff build uses the official BNU soil-category dataset when it creates the static file. The archive is not included in the case repository or Release.

DOWNLOAD AND EXTRACT

```
export MPAS_ROOT="$HOME/MPAS"
cd "$MPAS_ROOT/DATA/mpas_static"

curl -fL -O \
  https://www2.mmm.ucar.edu/projects/mpas/bnu_soiltype_top.tar.bz2

tar -xjf bnu_soiltype_top.tar.bz2
test -s "$MPAS_ROOT/DATA/mpas_static/bnu_soiltype_top/index"
```

The BNU dataset changes the static soil-category field; it does not replace GFS soil moisture or soil temperature. Guide 2 shows where to set config_soilcat_data = 'BNU'. With the standard hkust-dev v8.2.2 source, omit that option and retain STATSGO.

Ready for Guide 2

- init_atmosphere_model and atmosphere_model exist.
- The standard MPAS static dataset is available.
- mpas_static/mpas_cglc_lcz/index exists.
- For the pinned HRAIN2025 cutoff build, mpas_static/bnu_soiltype_top/index exists.

Appendix A | Build the MPAS dependencies from scratch

Use this appendix only when the server has no compatible MPI, NetCDF, or PnetCDF environment. Run A.2-A.10 in the same terminal session and use the same GNU compiler family and MPI wrappers for every MPI-linked package. The required build order is zlib, HDF5, NetCDF-C, NetCDF-Fortran, and then PnetCDF. If you open a new terminal, repeat the exports in A.2 and, when applicable, the final Open MPI exports in A.3 before continuing.

A.1 | Example versions

- Open MPI 4.1.6
- zlib 1.3.1
- HDF5 1.14.6
- NetCDF-C 4.9.3
- NetCDF-Fortran 4.6.4
- PnetCDF 1.12.3

These are pinned tutorial examples, not a requirement to use the newest release. Changing several versions at once makes failures harder to diagnose.

A.2 | Create the local prefix

```
export MPAS_ROOT="$HOME/MPAS"
export MPAS_PREFIX="$MPAS_ROOT/opt/mpas-libs"
export MPAS_LIB_SRC="$MPAS_ROOT/src/mpas-libs"
mkdir -p "$MPAS_PREFIX" "$MPAS_LIB_SRC"

export PATH="$MPAS_PREFIX/bin:$PATH"
export LD_LIBRARY_PATH="$MPAS_PREFIX/lib:$MPAS_PREFIX/lib64:${LD_LIBRARY_PATH:-}"
export CPATH="$MPAS_PREFIX/include:${CPATH:-}"
export LIBRARY_PATH="$MPAS_PREFIX/lib:$MPAS_PREFIX/lib64:${LIBRARY_PATH:-}"
export
PKG_CONFIG_PATH="$MPAS_PREFIX/lib/pkgconfig:$MPAS_PREFIX/lib64/pkgconfig:${PKG_CONFIG_PATH:-}"
export CPPFLAGS="-I$MPAS_PREFIX/include"
export LDFLAGS="-L$MPAS_PREFIX/lib -L$MPAS_PREFIX/lib64"

gcc --version
gfortran --version
make --version
```

PURPOSE MPAS_PREFIX is the single installation location. MPAS_LIB_SRC stores downloaded source code. PATH finds the installed configuration tools, while the include and library variables allow later configure, link, and runtime checks to find the same dependency stack.

A.3 | Optional user-space Open MPI

PURPOSE Build Open MPI only when the server does not already provide compatible mpicc, mpif90, and mpiexec commands. The wrappers printed at the end must resolve to the intended GNU compilers.

Skip this section when compatible mpicc, mpif90, and mpiexec commands already exist.

```
export MPAS_ROOT="$HOME/MPAS"
export MPI_PREFIX="$MPAS_ROOT/opt/openmpi-4.1.6"
mkdir -p "$MPAS_ROOT/src" "$MPI_PREFIX"
cd "$MPAS_ROOT/src"

curl -fL -O https://download.open-mpi.org/release/open-mpi/v4.1/openmpi-4.1.6.tar.gz
tar -xzf openmpi-4.1.6.tar.gz
cd openmpi-4.1.6

./configure --prefix="$MPI_PREFIX" CC=gcc CXX=g++ FC=gfortran
make -j 8
make install

export PATH="$MPI_PREFIX/bin:$PATH"
export LD_LIBRARY_PATH="$MPI_PREFIX/lib:${LD_LIBRARY_PATH:-}"
mpicc --showme:command
mpif90 --showme:command
```

A.4 | Build zlib

PURPOSE zlib supplies compression used by HDF5 and NetCDF. Continue only when make check passes and $MPAS_PREFIX/include/zlib.h exists.

```
cd "$MPAS_LIB_SRC"
curl -fL -O https://zlib.net/fossils/zlib-1.3.1.tar.gz
tar -xzf zlib-1.3.1.tar.gz
cd zlib-1.3.1
./configure --prefix="$MPAS_PREFIX"
make -j 8
make check
make install
test -s "$MPAS_PREFIX/include/zlib.h"
```

A.5 | Build HDF5

PURPOSE HDF5 supplies the NetCDF-4 storage layer. This tutorial needs the C library; the separate HDF5 Fortran interface is not required by NetCDF-Fortran.

```
cd "$MPAS_LIB_SRC"
curl -fL -O https://github.com/HDFGroup/hdf5/releases/download/hdf5_1.14.6/hdf5-1.14.6.tar.gz
tar -xzf hdf5-1.14.6.tar.gz
cd hdf5-1.14.6

CC=gcc ./configure \
  --prefix="$MPAS_PREFIX" \
  --with-zlib="$MPAS_PREFIX" \
  --enable-shared \
  --disable-fortran

make -j 8
make check
make install
h5cc -showconfig | head -n 12
```

A.6 | Build NetCDF-C

PURPOSE NetCDF-C supplies netcdf.h and libnetcdf. The final nc-config --has-nc4 check should print yes.

```
cd "$MPAS_LIB_SRC"
curl -fL -O https://downloads.unidata.ucar.edu/netcdf-c/4.9.3/netcdf-c-4.9.3.tar.gz
tar -xzf netcdf-c-4.9.3.tar.gz
cd netcdf-c-4.9.3
CC=gcc ./configure --prefix="$MPAS_PREFIX"   --disable-dap --enable-netcdf-4 --enable-shared
make -j 8
make check
make install
nc-config --version
nc-config --has-nc4
test -s "$MPAS_PREFIX/include/netcdf.h"
```

A.7 | Build NetCDF-Fortran

PURPOSE NetCDF-Fortran supplies netcdf.mod and libnetcdff for the Fortran MPAS source. It must be built against the NetCDF-C installation under the same MPAS_PREFIX.

```
cd "$MPAS_LIB_SRC"
curl -fL -O https://downloads.unidata.ucar.edu/netcdf-fortran/4.6.4/netcdf-fortran-4.6.4.tar.gz
tar -xzf netcdf-fortran-4.6.4.tar.gz
cd netcdf-fortran-4.6.4
CC=gcc FC=gfortran ./configure   --prefix="$MPAS_PREFIX" --enable-shared
make -j 8
make check
make install
nf-config --version
test -s "$MPAS_PREFIX/include/netcdf.mod"
```

A.8 | Build PnetCDF with the selected MPI wrappers

PURPOSE PnetCDF supplies parallel classic-NetCDF I/O. Build it with the same mpicc and mpif90 wrappers that will compile MPAS.

```
cd "$MPAS_LIB_SRC"
curl -fL -O https://parallel-netcdf.github.io/Release/pnetcdf-1.12.3.tar.gz
tar -xzf pnetcdf-1.12.3.tar.gz
cd pnetcdf-1.12.3
CC=mpicc FC=mpif90 ./configure --prefix="$MPAS_PREFIX"
make -j 8
make check
make install
pnetcdf-config --version
test -s "$MPAS_PREFIX/include/pnetcdf.h"
```

A.9 | Export the MPAS build variables

BACKEND CHOICE The three configuration tools identify the exact NetCDF-C, NetCDF-Fortran, and PnetCDF prefixes built above. Exporting all three variables also prevents values left by an older shell environment from being reused. PIO remains unset so the MPAS Makefile selects bundled SMIOL.

```
export PATH="$MPAS_PREFIX/bin:$PATH"
export LD_LIBRARY_PATH="$MPAS_PREFIX/lib:$MPAS_PREFIX/lib64:${LD_LIBRARY_PATH:-}"

export NETCDF="$(nc-config --prefix)"
export NETCDFF="$(nf-config --prefix)"
export PNETCDF="$(pnetcdf-config --prefix)"

# Empty PIO means: use the SMIOL source bundled with MPAS.
unset PIO

which mpicc mpif90 nc-config nf-config pnetcdf-config
mpicc --showme:command
mpif90 --showme:command
nc-config --version
nf-config --version
pnetcdf-config --version
```

Return to Step 2 only after A.10 passes. If any make check or prefix check fails, stop: continuing usually produces a harder-to-diagnose MPAS compile or runtime error.

A.10 | Confirm one consistent dependency stack

The following checks confirm that the NetCDF tools and headers come from MPAS_PREFIX and that both MPI wrappers use the intended compiler family.

```
test "$(nc-config --prefix)" = "$MPAS_PREFIX"
test "$(nf-config --prefix)" = "$MPAS_PREFIX"
test "$(pnetcdf-config --prefix)" = "$MPAS_PREFIX"

test -s "$NETCDF/include/netcdf.h"
test -s "$NETCDFF/include/netcdf.mod"
test -s "$PNETCDF/include/pnetcdf.h"

printf 'PIO=%s\n' "${PIO-<unset>}"

mpiexec --version
```

EXPECTED The three prefix tests return no output and exit successfully. The printed PIO value is <unset>, which confirms that MPAS will use bundled SMIOL. If any prefix differs, repair PATH and the A.2 exports before compiling MPAS.

A.11 | Common failures and what to check

- netcdf.mod not found - confirm that NetCDF-Fortran completed, $MPAS_PREFIX/include/netcdf.mod exists, and NETCDFF points to MPAS_PREFIX.
- Undefined references to nf_* or nc_* - NetCDF-C and NetCDF-Fortran were probably found from different prefixes, or the library path exports from A.2 were not restored in the current shell.
- pnetcdf.h, pnetcdf.mod, or MPI symbols missing - rebuild PnetCDF with the same mpicc and mpif90 wrappers selected for the MPAS build.
- PIO test or PIO library error - check whether a module or shell startup file set PIO. Leave it unset for the bundled SMIOL workflow used here.
- Shared library not found at runtime - restore LD_LIBRARY_PATH from A.2 before running the executables.